Computer understands: 0s and 1s -> binary... machine language low level- language

python, java, javascript, c++.... high level languages: human can understand them.. thy get compiled to machine udnerstandable form

Introduction to Programming & Python Basics

1. Welcome to Session 1: Getting Started with Programming!

Welcome to our AI and Programming Bootcamp! Today, we begin our journey into the exciting world of `coding.`

2. Icebreaker Activity

Let's get to know each other!

Please answer these questions one by one:

- What's your **name**?
- What's a **fun fact** about you?
- Why are you excited to learn programming?

Use `Shift + Enter` to run the cell below

```
name = input("What is your name? ")
fun_fact = input("Tell us a fun fact about you: ")
reason = input("Why are you excited to learn programming? ")

print("\n Welcome, " + name + "!")
print("Fun Fact: " + fun_fact)
print("Excited to Learn Programming Because: " + reason)
```

```
What is your name? Eyobed
Tell us a fun fact about you: I love football
Why are you excited to learn programming? I want to know more aobut
programming

 Welcome, Eyobed!
Fun Fact: I love football
Excited to Learn Programming Because: I want to know more aobut
programming
```

3. Bootcamp Expectations

Here are a few expectations to help us have a great learning experience together:

- **Listen actively** and participate
- **Ask questions**: there's no such thing as a silly or stupid question
- **Support each other**: we're all learning
- **Try things on your own** before asking for help
- **Stay focused** during our short time together
- **Have fun!**

Let's make the most of this opportunity together

4. Real-World Tech Examples: Why Learn Programming?

Think of these questions:

- **What is programming?**
- **What is programming language?**

Machine language: in terms of 0s and 1s

Compiler... high level to low level map... for every key work there is instruction attached.

3 words print input for

3 words... print, input, for, print input, print for, input for, print input for

Let's look at some amazing things that programming helps build:

TikTok Filters

- Use **Computer Vision** to detect your face and apply fun effects
- AI tracks your eyes, mouth, and head movement in real time!

ChatGPT

- Uses **Natural Language Processing** to understand and generate text
- Built using **Machine Learning** and trained on huge amounts of data

Everyday Examples

- Google Maps, Instagram, YouTube, and even simple calculators: all use programming!

All of these are built with **code**. And today, you'll start writing your own!

Let's begin with your **first Python program**!

5. Comments in Python

Comments are lines in your code that the computer **ignores**.

We use them to:

- Explain what the code does
- Leave notes for yourself or others
- Temporarily turn off lines of code

In Python, comments start with `#` for single line and `'''` for Multiline comment

Example:

```
# This line is a comment
print("Hello!")  # This prints a greeting

''' This is
multiline
comment
'''

# 5

# This is comment. it will not run

# I want to say how are you students
print("How are you students")

How are you students

print("Python is fun!") #this is a comment in the middle of code. It
will run.

Python is fun!

''' Multiline
comment
'''

{"type":"string"}
```

6. Your First Python Program

```
# Your very first Python program!

print("Hello, world!")

Hello, world!

print(13)

13

print("Ambaricho")

Ambaricho
```

Let's Try It!

Click the ▶ (Run) button above(at the top left) or press `Shift + Enter` to run the code.

This line of code(`print`) tells the computer to display a message. That's it!

Try changing the message to your name:

```python print("Hello, my name is _____!")

```
print("Hello, my name is winner")

Hello, my name is winner

#write your code here
print("Hello, my name is Selamu")

Hello, my name is Selamu

print(12)

12
```

# 7. Getting Input from the User

```
Asking the user for their name and greeting them!

name = input("What is your name? ")
print("Nice to meet you, " + name + "!")

What is your name? winner
Nice to meet you, winner!

#write a program that asks for your age
input("How old are you?")

How old are you?10

{"type":"string"}

#ask for name
input("What is your name? ")
#ask for age
input("How old are you? ")

What is your name? Eyobed
How old are you? 10

{"type":"string"}

input("what your name? ")

what your name? Eyobed
```
```

```
{"type":"string"}
```

Variables are like containers

```
3

3

x = 3

print(x)

3

y = "hello"

print(y)

hello

name = "Ephraim"
father = "Tamiru"

print("Dereje Zeleke")

Dereje Zeleke

print(name, father)

Ephraim Tamiru

print(name + " " + father)

Dereje Zeleke

#ask your user for their name and father's name, then print it

name = input("What is your name? ")
father = input("what is your father's name? ")
print("Welcome " + name + " " + father)

What is your name? Ephraim
what is your father's name? Tamiru
Welcome Ephraim Tamiru

print(father)

Admasu
```

Getting to Know You (with `input()`)

The `input()` function lets the user type something. Python waits until they enter a value.

Try editing the message and ask for something else, like your favorite color:

```python color = input("What is your favorite color? ") print("Cool! I like " + color + " too.")

```
input("how old are you? ")

how old are you? 16

{"type":"string"}

#write your code here
color = input("What is your favorite color?")

print(color)
```

# 8. Python Data Types

Everything in Python has a **type**.

Here are some of the most common types:

| Type | Example | What It Represents |
|---|---|---|
| `int` | `5`, `100`, `-3` | Whole numbers |
| `float` | `3.14`, `2.0`, `-1.5` | Numbers with decimals |
| `str` | `"hello"`, `'7'`, `"42.0"` | Text (strings) |
| `bool` | `True`, `False` | Yes/No or On/Off |
| `list` | `[1, 2, 3]`, `["a", "b", "c"]` | A collection of items (like a box) |
| `None` | `None` | "Nothing" or "empty" (but still valid) |

Remember there are more data types like tuples, dictionaries...!

```
Examples of different data types

a = 10   # int
b = 3.14   # float
c = "hello"   # str
d = True   # bool
e = [1, 2, 3, 5, "hello"]   # list
f = None   # None
g = "True"   # str

print("a:", a, "->", type(a))
print("b:", b, "->", type(b))
print("c:", c, "->", type(c))
print("d:", d, "->", type(d))
print("e:", e, "->", type(e))
print("f:", f, "->", type(f))
```
```

```
a: 10 -> <class 'int'>
b: 3.14 -> <class 'float'>
c: hello -> <class 'str'>
d: True -> <class 'bool'>
e: [1, 2, 3, 5, 'hello'] -> <class 'list'>
f: None -> <class 'NoneType'>
```

`Challenge: Guess the Type!`

Look at the values below and **try to guess what type** each one is.

Then run the code to check your answers!

Write your guess as a comment using `#` before each line.

```
# Guess what type each variable is before running!

x = "42"          # Your guess: ____str___
y = 42            # Your guess: ___int____
z = 3.14          # Your guess: ___float____
happy = True      # Your guess: ___bool____
items = [1, 2, 3]# Your guess: ___list____
nothing = None    # Your guess: ___None____

print(type(x))
print(type(y))
print(type(z))
print(type(happy))
print(type(items))
print(type(nothing))
```

```
<class 'str'>
<class 'int'>
<class 'float'>
<class 'bool'>
<class 'list'>
<class 'NoneType'>
```

```
#1. ask the user for their grade
grade = input("What grade are you? ")
#2. ask the user for thier school
school = input("Enter your school name: ")
#3. print their grade and their school
```

```
What grade are you? 11
Enter your school name: Higa
```

```
print(grade)
print(school)
```

```
11
Higa
```

Important Note about inputs in python: **All Inputs Are Strings**

When you use `input()`, Python treats **everything** you type as a `str` (string), even if you type a number!

Try this: ```python age = input("Enter your age: ") print(type(age))

```
age = input("How old are you? ")
print(age)

How old are you? 17
17

type(age)  #it is supposed to be int

str

type(age)

str

#let's ask the user true or false question
result = input("Tomato is a fruit(True/False): ")

Tomato is a fruit(True/False): True

type(result)

str

grocery = input("List your groceries:")

List your groceries:["avocado", "banada"]

type(grocery)

str
```

Why is this a problem?

- We can't do math with a string

```
age = input("Enter your age: ")
age + 5  # This causes Error! Can't add number to a string.

Enter your age: 17

---------------------------------------------------------------------------
-----
TypeError                                 Traceback (most recent call
```

```
last)
/tmp/ipython-input-679878326.py in <cell line: 0>()
      1 age = input("Enter your age: ")
----> 2 age + 5  # This causes Error! Can't add number to a string.

TypeError: can only concatenate str (not "int") to str
```

9. Fixing It with Type Casting

To use the number properly, we need to **convert** (cast) it:

```
age = int(input("Enter your age: "))  # here the int() will change the input type from str to int
print("In 5 years, you will be", age + 5)

age = "17"  #int("17")
a=int(age) #17
type(a)

int

result = input("Tomato is a fruit.(True/False)")
type(result)

Tomato is a fruit.(True/False)True

str

bool(result)

True

a = 3.89 #float
b = 4 #int

#convert a to int
int(a)
#convert b to float
float(b) #

4.0

10 + 5 # int + int

15

"10" + "5" #str + str 105

{"type":"string"}

int("10") + 5 #int + int
```

```
15

x = int("12") #12
y = float("3") #3.0
print(x + y) #12 + 3.0

15.0

age = input("HOw old are you? ") #str
print(age)

HOw old are you? 17
17

# This time we cast input to int so we can do math

age = int(input("Enter your age: "))
age + 5 #int + int

Enter your age: 17

22

float(2)

2.0

str(2)

{"type":"string"}

int(4.7)

4

x = -2.5 #float because it has decimal places
int(x)

-2

x = -2.5
```

10. Exploring Data Types with type()

Using the **`type()`** keyword will tell you the type of something

```
# Let's look at the types of our variables
b = 4.8
type(b)

float

type(1.0)
```

```
float

type(True)

bool

type([0,1,2,3])

list
```

11. Variables

- Python variables are not like math variables
- They can be assigned and hold a value (They are like containers to hold values)
- `=` is used to assign a value to a varaible like: `variable_name = value`

For example: ```python x = 1

#means there is container(variable) named x and it contains 1

```
x = 2
#putting variable name at the end of the block will print it
x

2

#check the type of the variable
type(x)

int

#we can reassign a variable
x = 3
x

3

x = True
x

True

kebron = 4.8
int(4.8)

4

x = 2
x = 3
x = False
```

```
x = "Hello"
print(x)
```

```
Hello
```

```
#the name variable is not related to the value(it is totally up to you
to choose the name)
two = 2
two
```

```
2
```

```
two = 3
two
```

```
3
```

```
#variables are case sensitive(upper case and lower case matters)
X = 4.
X
```

```
4.0
```

```
x #remember x(lower case) is 3, and X(upper case) is 4.
```

```
{"type":"string"}
```

```
#variables name should be exact
Xyz = 3
Xyz
```

```
3
```

```
xyz =4
xyz
```

```
4
```

```
Xyz #pay attention to X
```

```
3
```

12. Operations(Integral Types)

Once we have numbers (like `int` or `float`), we can do math!

Here are list of arthemetic operations:

Operator	Symbol	Example	Description
Add	`+`	`3 + 2`	Adds two numbers
Subtract	`-`	`5 - 1`	Subtracts two

Operator	Symbol	Example	Description
			numbers
Multiply	`*`	`4 * 2`	Multiplies
Divide	`/`	`8 / 2`	Division (decimal)
Floor Division	`//`	`9 // 2`	Division (whole only)
Modulo	`%`	`10 % 3`	Remainder (modulus)
Exponent	`**`	`2 ** 3`	2 to the power of 3

```
1+1
```

```
2
```

```
#the value of arthemtic operation can be put into a variable
x = 1+1
x
```

```
2
```

```
y = 10 - 4.3
y
```

```
5.7
```

```
#the result of arthemetic operation WILL NOT CHANGE the variable
contents
y = y*2
```

```
print(y)
```

```
11.4
```

```
y
```

```
11.4
```

```
5 + 5.0
```

```
10.0
```

```
#as you can see, variable y is unchanged
y
```

```
5.7
```

```
#we have to reassign the value to the varialbe inorder to update it
y = y*2
y
```

```
11.4
```

```
#we can make exponent of a number by using ** operation
4**2 # it means 4 the power of 2
```

```
16
```

```
4**3
```

```
64
```

```
#float division
4.0/2.0
```

```
2.0
```

```
# division of integers
4/3
```

```
1.3333333333333333
```

```
#if we want only the integer part of division, we use //
# integer division
4 // 3
```

```
1
```

```
# guess the output
7 // 3
```

```
2
```

```
10 // 3
```

```
3
```

```
4.0 // 3.0
```

```
1.0
```

```
# to find reminder of division we use "modulus" or "mod". it's symbol
is %
4 % 3
```

```
1
```

```
# # guess
17 % 3
```

```
2
```

```
10 % 4
```

```
2
```

```
20 % 4
```

```
0
```

```
143 % 5
```

```
3
```

Short forms of operations when using same variable

```
x = 1 #1 is assigned to the variable x
x = x + 1 #we added 1 on x and assigned it back to x
x #2
```

```
2
```

```
x = 1
x += 1 #This is equivalent to "x = x + 1"
x
```

```
2
```

```
x = 1
x + 1
```

```
2
```

```
x
```

```
1
```

```
x = 10
x -= 4 # This is equivalent to x = x - 4
x
```

```
6
```

```
x = 60
x *= 50 # This is equivalent to _x = x * 50_________
x
```

```
3000
```

```
x = 100
x /= 10 # This is equivalent to _____x = x/10______
x
```

```
x = 100
x //= 6 # This is equivalent to ____x = x // 6________
x
```

```
16
```

```
x = 2
y = 3
x = y # = is only for variable assignment
x
```

```
3
```

```
# '==' is used to check if two variables or values are equal
x = 10
y = 10
x == y
```

```
True
```

```
x = 10
y = 20
x == y
```

```
False
```

Order of operations matters!!

*, /, % happen first, in the order they appear. + and - are the last

```
1 + 3 * 20 % 7
# 1 + 60 % 7
# 1 + 4
# 5
```

```
5
```

```
(1 + 3) * 20 % 7
# 4 * 20 % 7
# 80 % 7
# 3
```

```
3
```

```
5*3-4%3
```

13. Operations(boolean types)

Python can compare values and give answers like `True` or `False`.

Comparison Operators:

Operator	Symbol	Example	Result
Equals	`==`	`5 == 5`	`True`

Operator	Symbol	Example	Result
Not Equal	`!=`	`5 != 3`	`True`
Greater	`>`	`7 > 3`	`True`
Less	`<`	`4 < 2`	`False`
Greater or Equal	`>=`	`5 >= 5`	`True`
Less or Equal	`<=`	`2 <= 3`	`True`

Logical Operators:

Operator	Keyword	Example	Result
AND	`and`	`5 > 3 and 2 < 4`	`True`
OR	`or`	`5 < 3 or 2 < 4`	`True`
NOT	`not`	`not (5 > 3)`	`False`

- `and`: Both sides must be `True`
- `or`: At least one side must be `True`
- `not`: Flips the value (`True` → `False`, `False` → `True`)

```
# Try Boolean comparisons

a = 5
b = 3

print("a == b:", a == b) #False
print("a != b:", a != b) #True
print("a > b:", a > b) #True
print("a < b:", a < b) #False

a == b: False
a != b: True
a > b: True
a < b: False

# Logical operators in action

a = 5
b = 3

print("a > 2 and b > 2:", a > 2 and b > 2) # True and True = True
print("a > 2 and b > 5:", a > 2 and b > 5) # True and False = False
print("a < 2 or b > 2:", a < 2 or b > 2) # False or True = True
print(a < 2 or b < -10) #False or False = False
print("not a > 2:", not a > 2) # not True = False

a > 2 and b > 2: True
a > 2 and b > 5: False
```

```
a < 2 or b > 2: True
not a > 2: False
```

Challenge: Who Gets a Discount?

Imagine you're building a program for a movie theater.

They want to give **a discount** if:

- The person is **under 18 years old**, OR
- The person **is a student**

Your task:

1. Ask the user for their age
2. Ask if they are a student (answer: "yes" or "no")
3. Decide if they get a discount using **Boolean logic**
4. Print `True` if they get the discount, `False` if not

Use `or`, `==`, and `int()` for this challenge.

Can you figure it out? Try it in the code cell below!

```
# Write your solution here!

# Step 1: Get age
age = int(input("How old are you? "))
# Step 2: Get student status
status = input("Are you student?(yes/no)")
```

```
How old are you? 20
Are you student?(yes/no)Yes
```

```
#check if age is less than 18
age < 18
```

```
False
```

```
status == "yes"
```

```
True
```

```
age < 18 or status == "yes"
```

```
False
```

```
a = "Ethiopia"
#change it to lower case
a.lower()
```

```
{"type":"string"}

a = "ethiopia"
# change it to upper case
a.upper()

{"type":"string"}

# Write your solution here!

# Step 1: Get age
age = int(input("How old are you? "))
# Step 2: Get student status
status = input("Are you student?(yes/no)")

How old are you? 20
Are you student?(yes/no)YeS

status.lower() == "yes"

True

status.upper() == "YES"

True

#check if status is yes
status.lower() == "yes"
```

how do we combine them? which logical operation do we use? "or" or "and"

```
#write your code here

#read the age and status
age = int(input("Enter your age: "))
student = input("Are you a student? (yes/no): ")

# Convert student answer to boolean
is_student =
age_check =

# Check discount condition
can_get_discount =
print("Eligible for discount:", can_get_discount)

45//4

45 % 4
```

14. String and List Operations

Python lets us **combine and repeat** strings and lists!

Strings:

```
"Hello" + "World"    # Combines (concatenates)
"Hi " * 3            # Repeats 3 times (Hi Hi Hi)
```

Lists:

```
[1, 2] + [3, 4]      # Combines two lists
["a"] * 5            # ['a', 'a', 'a', 'a', 'a']

# Working with strings
first = "Hello"
second = "World"
print(first + " "+ second)  # Concatenation

Hello World

# Repeating
laugh = "ha"
print(laugh * 5)

hahahahaha

# Working with lists
numbers = [1, 2, 3]
print(numbers + [4, 5])  # Add two lists

[1, 2, 3, 4, 5]

print(["wow", "hello"] * 3)       # Repeat a list item

['wow', 'hello', 'wow', 'hello', 'wow', 'hello']
```

15. What Happens When Something Goes Wrong?

Sometimes your code will crash or break and that's okay! Python will give you an **error message**.

Common error types:

- `SyntaxError` → You typed something wrong
- `TypeError` → You used the wrong type (e.g. adding string to int)
- `NameError` → You used a variable that doesn't exist
- `ZeroDivisionError` → You tried to divide by 0

Read the error message: it usually tells you **what** and **where** the problem is.

```
# Uncomment each line one by one and run to see the error

# print("hello"
# 5 + "5"
```

```
# print(w)
# print(1 / 0)

[1, 2] * [3,4]
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipython-input-2598083740.py in <cell line: 0>()
----> 1 [1, 2] * [3,4]

TypeError: can't multiply sequence by non-int of type 'list'
```

```
x * 2
```

```
-5.0
```